

Esculturas que Engañan a la Luz: Replicación y Evaluación de Shadow Art mediante Renderizado Diferenciable

Motivación

El "shadow art" es una escultura computacional donde un único objeto 3D proyecta sombras distintas según la dirección de iluminación. La pregunta que buscamos responder es: ¿Qué tan bien generaliza el método de Sadekar et al. (WACV 2022) para reconstruir un modelo 3D completo usando siluetas de distinta complejidad geométrica, y cómo afecta la elección de representación (voxel vs. mesh) y la cantidad de fotos usadas en la calidad del resultado?

Método base y trabajo realizado

El método del paper original ([Shadow Art Revisited: A Differentiable Rendering Based Approach, WACV 2022](#)) optimiza mediante un descenso de gradiente sobre un renderizador diferenciable la geometría de un objeto 3D para que sus proyecciones de sombra bajo distintas configuraciones de luz coincidan con un conjunto de imágenes binarias objetivo. Se ejecutará el pipeline completo en ambos modos disponibles (voxel y mesh) sin modificar la arquitectura base del método.

La contribución principal del proyecto es una replicación del paper base con profundización en datos y evaluación.

Se planteó una evaluación adicional a la propuesta en el paper original, profundizando en las métricas existentes (como IoU), comparando los resultados para un mismo modelo según la cantidad de imágenes usadas para confeccionarlo, entregando una perspectiva nueva sobre las capacidades de este método junto a sus limitaciones.

Datos

Se utilizó como base el dataset del repositorio oficial del paper: ShadowArt-Revisited ([kaustubh-sadekar/ShadowArt-Revisited](#)), el cual se encuentra en la ruta "data/viewsdataset", en donde hay 15 siluetas binarizadas y en formato .PNG (duck.png, mikey.png, batman.png, etc.) que en total suman 221KB. El grupo contó con acceso completo a este dataset durante el desarrollo del proyecto y las imágenes no requirieron preprocesamiento al estar ya binarizadas. De estas 15 siluetas, se decidió usar solo 9 de ellas (mikey, puma, spiderman, superman, batman, bunny_0, teddy2, duck y heart) para replicar los resultados mostrados en la figura 4 del paper original, en donde se muestran las métricas de IoU y Dice para las sombras generadas por el modelo 3D obtenido en 4 combinaciones de siluetas distintas, de esta manera, las métricas calculadas a través de esta replicación servirán como baseline para evaluar la generalización del modelo para el resto de figuras nunca antes vistas por el pipeline de Mesh y Voxel.

Luego, para evaluar la generalización se binarizaron 10 imágenes de openclipart.org (licencia CC0) con el siguiente pipeline: (1) extracción de máscara por canal alfa cuando disponible, o umbralización de Otsu sobre la escala de grises en caso contrario, además se realiza una inversión automática para imágenes con figura oscura sobre fondo claro (3) limpieza morfológica con apertura y cierre para eliminar artefactos (4) recorte al bounding box de la figura con margen del 15% y reescalado a 512×512 píxeles. El resultado son imágenes estrictamente binarias (0 o 255) en el mismo formato que el dataset original del paper. Finalmente, se generaron 5 siluetas adicionales desde cero usando las librerías NumPy y CV2.

Estas 15 siluetas nuevas varían deliberadamente su complejidad geométrica para evaluar la generalización del método fuera del dataset original, con 5 imágenes con un nivel de complejidad geométrica simple, 5 media y 5 alta. En donde la "complejidad geométrica" se definió a través del valor de compacidad de las siluetas, con la fórmula: $compacidad = \frac{4 \cdot \pi \cdot area}{perimetro^2}$, tal que: Simple: [0.42, 1], Media: [0.23, 0.42) y Alta: [0.02, 0.23). De esta manera, al clasificar las 9 imágenes seleccionadas del dataset original con esta definición de complejidad geométrica se obtuvo:

Complejidad	Silueta	Área	Compacidad	Aspecto
Simple	heart	18.3%	0.7900	1.15
Simple	duck			
Media	bunny_0			
Media	mikey	17.6%	0.4182	1.07
Media	teddy2	15.5%		
Media	batman	11.6%		
Alta	superman	11.3%	0.2049	1.00
Alta	puma	8.4%	0.2020	1.45
Alta	Spider-Man	8.4%	0.1632	1.15

Tabla 1: Clasificación de complejidad geométrica para las 9 imágenes seleccionadas

Luego, para las 15 figuras del dataset generado por nosotros se tiene lo siguiente:

Complejidad	Silueta	Área	Compacidad	Aspecto
Simple	s1_circulo	23.0%	0.9092	1.00
Simple	s2_rectangulo	18.7%		1.55
Simple	s3_cruz	18.5%		1.00
Simple	s4_flecha	10.1%		1.41
Simple	s5_house	11.0%		1.74
Media	m1_bird	7.9%	0.4196	1.86
Media	m2_tree	14.2%		1.05
Media	m3_fish	7.8%		1.96
Media	m4_star	10.0%		1.06
Media	m5_guitar	4.1%		3.03
Alta	h1_sitting_cat	9.9%	0.2240	1.11
Alta	h2_running_person	5.3%		2.34
Alta	h3_butterfly	7.6%		1.50
Alta	h4_snowflake	11.7%	0.0794	1.13
Alta	h5_bike	3.9%	0.0226	1.74

Tabla 2: Clasificación de complejidad geométrica para el dataset construido

A partir de estas dos tablas, se observan dos limitaciones principales sobre el dataset que construimos:

1. La primera es la relación de aspecto de nuestras imágenes. Mientras que en las 9 imágenes del dataset original se observa que la relación de aspecto nunca va más allá del 1.5 (1.45 máximo, para la imagen puma), en nuestras imágenes el promedio de esta característica es de 1.5653, en donde la imagen "m5_guitar" posee el máximo con 3.03. Esto, representa una limitación debido a la forma en que funciona el pipeline de shadow art de los autores, el cual busca la intersección 3D del volumen, por lo tanto, al intentar cruzar la silueta de la guitarra (que es muy horizontal), con una persona corriendo (vertical), se observará que el modelo 3D se verá limitado a la sección en donde ambas siluetas coinciden, viéndose obligado a descartar el resto de la información.
2. La segunda corresponde a la cantidad de masa (área) de las figuras m5_guitar, h2_running_person y h5_bike, las cuales se alejan notoriamente del área mínima del conjunto original seleccionado (3.1% de diferencia entre el mínimo de puma y el máximo de h2_running_person), lo cual representa un problema debido a las funciones de suavizado del optimizador de PyTorch3D que usan los autores, las cuales, podrían afectar negativamente las métricas al erosionar las partes más delgadas de estas 3 siluetas.

Sobre los datos necesarios para evaluar las capacidades del pipeline para reconstruir modelos 3D se decidió hacer uso de assets públicos provenientes del sitio web [Free3D](#). Los modelos extraídos fueron visualizados mediante Blender para obtener sus siluetas de acuerdo a distintos puntos de vista en formato de imágenes .png, todas con una resolución de 512x512 píxeles presentando la forma del objeto completamente blanca sobre un fondo negro, estas siluetas corresponden, por simplicidad, a proyecciones ortogonales de los modelos, vistas desde sus secciones laterales junto a perspectivas $\frac{3}{4}$ de los mismos, conformando en total 8 imágenes de no más de 90KB en conjunto por cada modelo. Debido a limitaciones de tiempo de cómputo se escogieron 2 modelos para analizar, correspondiendo a un auto el cual destaca por tener una figura simple y simétrica, mientras que el segundo es un rifle de asalto, que tiene una forma más compleja y menos uniforme en comparación.

Evaluación

Para la primera parte de la evaluación de este proyecto, se usaron 2 métricas definidas por los autores: Intersection over Union (IoU) (función *iou_np*) y Coeficiente de Similitud de Dice (función *dice_np*), junto a 3 métricas definidas por nosotros: ROI Pixel Accuracy (función *_roi_pixel_accuracy*), Precision y Recall. A partir de esto, las fórmulas que definen a cada una de estas métricas son:

Fórmula	Descripción
$IoU = \frac{\Sigma(Pred \bullet Target)}{\Sigma(Pred + Target - Pred \bullet Target)}$	Mide el porcentaje de superposición entre la sombra generada y la silueta real, penaliza fuertemente cualquier deformidad o desfase en los bordes de la sombra obtenida
$Dice = \frac{2\Sigma(Pred \bullet Target)}{\Sigma Pred + \Sigma Target}$	Mide la similitud espacial general entre la predicción y el objetivo, a diferencia de la métrica anterior, es ligeramente más indulgente con los errores.
$ROI_PA = \frac{TP + FN}{TP + TN + FP + FN}$	Indicador de precisión clasificatoria por píxel, el cual se cuantifica considerando únicamente la zona operativa de la silueta, descartando los componentes del entorno que no tienen relevancia. En donde TP, TN, FP y FN corresponden a True Positive, True Negative, False Positive y False negative, respectivamente.
$Precision = \frac{TP}{TP + FP}$	Evalúa la confiabilidad de los píxeles dibujados, indicando qué proporción de la sombra generada por el modelo pertenece a la figura objetivo, castigando la existencia de "masa extra".

$Recall = \frac{TP}{TP+FN}$	Evalúa la completitud de la proyección, indicando qué porcentaje de la silueta real logró ser cubierta correctamente por el modelo, castigando omisiones.
-----------------------------	---

Formulas 1, 2, 3, 4 y 5: IoU, Dice, ROI_PA, Precision y Recall, respectivamente

Con estas métricas, se generó el baseline de la generalización de los métodos (Voxel y Mesh) a partir de las combinaciones mostradas en la figura 4 del paper original, y se obtuvieron los siguientes resultados:

Experimento	Silueta	Método	IoU	Dice	Precision	Recall	ROI_PA
2-mikey-puma	mikey	Voxel	0,9893	0,9946	0,9994	0,9899	0,9949
		Mesh	0,9897	0,9948	0,9925	0,9972	0,995
	puma	Voxel	0,983	0,9914	0,997	0,9859	0,9933
		Mesh	0,9831	0,9915	0,9891	0,9938	0,9934
3-heroes	Spider-Man	Voxel	0,9467	0,9726	0,9856	0,9599	0,9831
		Mesh	0,8827	0,9377	0,9108	0,9663	0,9599
	superman	Voxel	0,8417	0,914	0,996	0,8445	0,942
		Mesh	0,8851	0,9391	0,956	0,9227	0,9567
	batman	Voxel	0,9314	0,9645	0,9817	0,9479	0,9783
		Mesh	0,8945	0,9443	0,9439	0,9447	0,9585
3-bunny-teddy-duck	bunny_0	Voxel	0,9618	0,9806	0,973	0,9882	0,9791
		Mesh	0,9657	0,9826	0,9708	0,9946	0,9796
	teddy2	Voxel	0,9446	0,9715	0,9991	0,9454	0,962
		Mesh	0,9821	0,991	0,9892	0,9927	0,9876
	duck	Voxel	0,9886	0,9943	0,9991	0,9895	0,9929
		Mesh	0,9807	0,9902	0,9854	0,9951	0,9878
3-mikey-puma-heart	mikey	Voxel	0,9655	0,9824	0,9965	0,9688	0,9834
		Mesh	0,9575	0,9783	0,964	0,993	0,9789
	puma	Voxel	0,9566	0,9778	0,9679	0,988	0,9826
		Mesh	0,9439	0,9711	0,9516	0,9915	0,9773
	heart	Voxel	0,9619	0,9806	0,9995	0,9624	0,9739
		Mesh	0,9857	0,9928	0,9926	0,9931	0,9901

Tabla 3: Baseline para las comparaciones

A partir de estos resultados, es posible observar que las métricas de IoU y de Dice de cada experimento (en ambos métodos) no coinciden de forma exacta con los resultados mostrados en la figura 4 del paper a pesar de haber usado los mismos parámetros y código que los autores, tanto para el baseline de Voxel como el de Mesh. Estas diferencias, que para algunas siluetas resulta positiva y para otras negativa, se atribuyen a dos razones:

- 1) La primera es la antigüedad del paper original, desde su publicación en 2022, Python ha avanzado de la versión 3.8 a la versión 3.12, NumPy de la versión 1.19 a la 1.26, y PyTorch3D de la versión 0.4.0 a la 0.7.1. Por lo tanto, se infiere que las optimizaciones subyacentes implementadas en estas librerías clave inciden directamente en la convergencia

del modelo, generando las variaciones positivas que se observan en múltiples siluetas del baseline.

- 2) La segunda es la ausencia de un control de aleatoriedad (fijación de semillas o seeds) en el código del paper, ya que, al no establecer un comportamiento determinista para las operaciones de los núcleos CUDA en PyTorch, ni para la inicialización del espacio volumétrico, el algoritmo termina obteniendo valores distintos entre cada ejecución realizada. Valores los cuales, pueden llegar a presentar variaciones significativas debido a la naturaleza acumulativa del optimizador Adam (torch.optim.Adam). Esto imposibilita una reproducibilidad matemática exacta, y creemos que también explica el hecho de que en algunas siluetas las métricas del baseline se vean afectadas de forma negativa en comparación a su contraparte del paper original.

Es debido a esto, que se usarán los resultados mostrados en la Tabla 3 como baselines del proyecto en vez de los resultados mostrados en la figura 4 del paper, de esta manera, se asegura la consistencia en el trabajo realizado.

Como siguiente paso en la evaluación de la generalización, se implementó una batería de cuatro “experimentos espejo”. Esta metodología busca recrear los escenarios de prueba expuestos en la Figura 4 del paper original, utilizando exclusivamente nuestro dataset de siluetas, las cuales fueron emparejadas para asemejarse lo más posible en la compacidad y complejidad geométrica utilizada por los autores. Para lo cual, se obtuvieron los siguientes resultados:

Experimento	Siluetas	Método	IoU	Dice	Precision	Recall	ROI_PA
2a-media-alta	m1_bird	Voxel	0,9897	0,9948	0,9983	0,9914	0,9951
		Mesh	0,9908	0,9954	0,9936	0,9972	0,9956
	h2_running_person	Voxel	0,9799	0,9899	0,9982	0,9816	0,9922
		Mesh	0,9209	0,9588	0,928	0,9918	0,9725
3b-1media-2alta	m5_guitar	Voxel	0,9045	0,9499	0,9662	0,9341	0,964
		Mesh	0,8742	0,9329	0,8885	0,9819	0,9435
	h1_sitting_cat	Voxel	0,6595	0,7948	0,9353	0,6911	0,8921
		Mesh	0,7492	0,8566	0,7776	0,9536	0,8921
	h3_butterfly	Voxel	0,7563	0,8612	0,9494	0,7881	0,9172
		Mesh	0,6932	0,8188	0,7217	0,9462	0,8617
3c-2simple-1media	s5_house	Voxel	0,8856	0,9393	0,9993	0,8862	0,93
		Mesh	0,981	0,9904	0,9981	0,9829	0,9884
	s4_flecha	Voxel	0,9538	0,9763	0,9675	0,9853	0,9785
		Mesh	0,9316	0,9646	0,9336	0,9978	0,9671
	m3_fish	Voxel	0,9766	0,9882	0,9957	0,9807	0,9885
		Mesh	0,9348	0,9663	0,9401	0,9939	0,9659
3d-simple-media-alta	s2_rectangulo	Voxel	0,8758	0,9338	1	0,8758	0,8825
		Mesh	0,9855	0,9927	0,9933	0,9921	0,9885
	m1_bird	Voxel	0,9792	0,9895	0,9839	0,9952	0,99
		Mesh	0,9095	0,9526	0,9115	0,9976	0,9606
	h2_running_person	Voxel	0,9623	0,9808	0,9744	0,9873	0,9851
		Mesh	0,8535	0,9209	0,8589	0,9927	0,9345

Tabla 4: Métricas experimentos espejo de generalización

En primer lugar, al comparar los resultados entre el baseline de Voxel y su contraparte de espejo (Tablas 3 y 4), se puede observar que el experimento 2a-media-alta logra generalizar el problema de forma correcta, ya que se observa que los resultados de esta combinación de siluetas son prácticamente idénticos a los del baseline a pesar de usar siluetas completamente diferentes (pero con una complejidad geométrica parecida), con diferencias de apenas 0.0014, 0.0006, 0.0001, 0.0014 y 0.0004 en las métricas promedio de IoU, Dice, Precision, Recall y ROI_PA, respectivamente.

Luego, al incrementar la restricción geométrica incorporando una tercera silueta (experimento 3b-1media-2alta), se revela la fragilidad del modelo bajo estrés. En el baseline, el IoU promedio experimentó una caída moderada al pasar de 0.9862 a 0.9066 (-8%). Mientras que, los experimentos espejo sufrieron un desplome mucho más drástico, pasando de un IoU promedio de 0.9848 a 0.7734 (-21%). Esta disminución en el rendimiento confirma la vulnerabilidad geométrica planteada anteriormente en las limitaciones de los datos: la combinación de una silueta extremadamente asimétrica como m5_guitar (con una relación de aspecto de 3.03) con figuras complejas como h1_sitting_cat y h3_butterfly provoca una pérdida masiva de información en el resultado final, debido al hecho de que el modelo se encuentra acotado a la intersección tridimensional donde estas tres proporciones logran coincidir.

A partir de esto último, inicialmente se observa un resultado contra-intuitivo en este experimento espejo a partir de las métricas de Recall, Las sombras que pierden más masa son las siluetas más compactas y de menor relación de aspecto (contenidas en el espacio dictado por la guitarra), en vez de la silueta con mayor relación de aspecto (la guitarra). Esto se puede confirmar visualmente a través de la siguiente figura que muestra la diferencia entre la sombra generada por el modelo 3D obtenido y la silueta original objetivo:

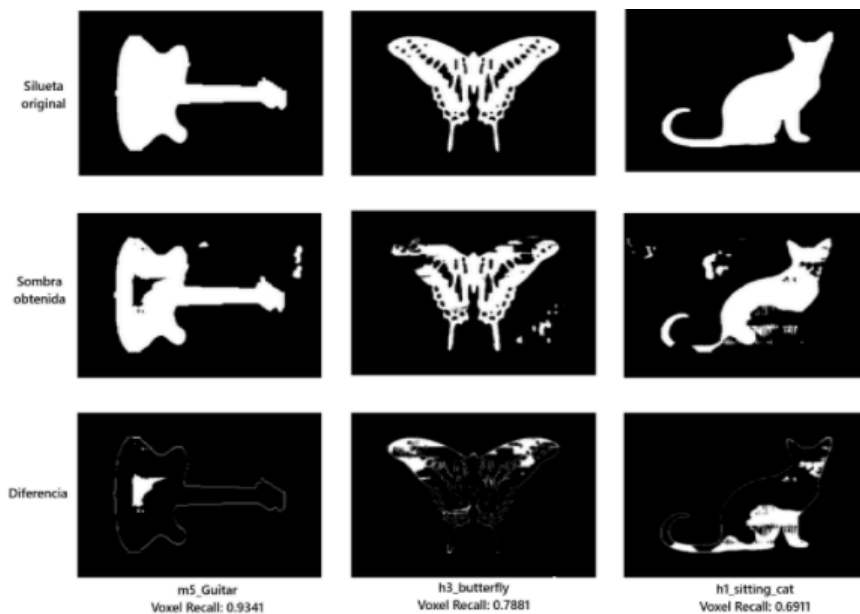


Figura 1: Resultados visuales experimento 3b-1media-2alta, método Voxel

Este fenómeno se puede explicar sobre la base de la misma limitación descrita anteriormente, la cual, en este caso, se combina con el déficit del área de la guitarra, que, al representar sólo un 4.1% de la imagen (Tabla 2), obliga al optimizador a perforar y eliminar casi la totalidad de los vóxeles a lo largo de su eje ortogonal, es decir, el 95.9% restante de la imagen. Esta sustracción actúa como una máscara destructiva, y se ve amplificada por la relación de aspecto irregular de este instrumento. Por lo tanto, al perfilar los bordes de esta silueta, el algoritmo termina erradicando la masa central que

resultaba indispensable para la conformación del resto de figuras más compactas y de mayor área (7.6% butterfly y 9.9% sitting_cat, Tabla 2).

Luego, en los experimentos 3 y 4 del método de Voxel, se puede observar nuevamente que la caída de rendimiento es mayor en contraste a su contraparte en el baseline, sin embargo, a diferencia del experimento anterior, la disminución no es tan drástica (-2.4% para IoU y -2.2% para Recall). No obstante, al analizar los resultados individuales de las figuras en lugar de sus promedios, emerge un patrón llamativo: las siluetas de complejidad simple, que teóricamente deberían ser las más fáciles de reconstruir, sufren caídas mayores en contraste con aquellas clasificadas como más difíciles. Por ejemplo, s5_house y s2_rectangulo presentan un IoU de 0.8856 y 0.8758 respectivamente, mientras que las figuras de mayor dificultad que las acompañan, como m3_fish y h2_running_person, logran mantener métricas destacables (IoU > 0.96 y Recall > 0.98).

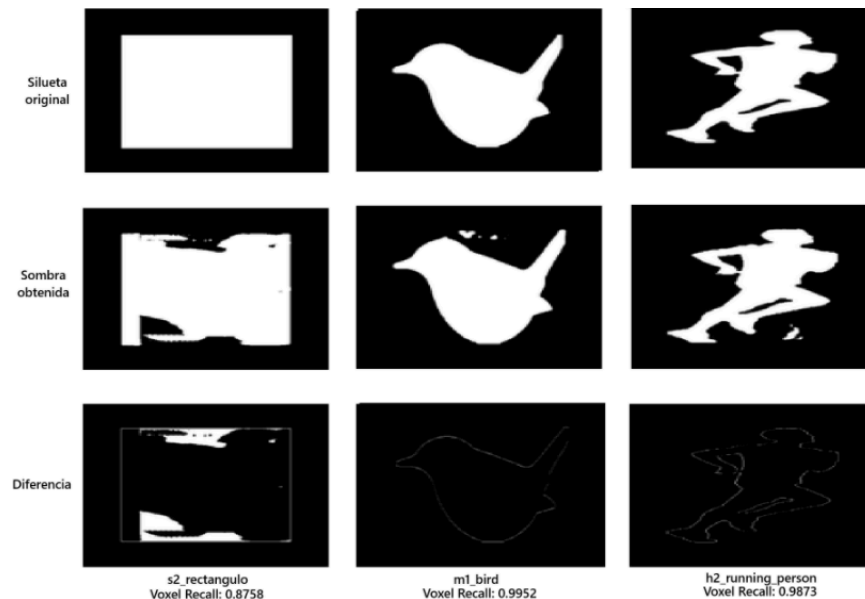


Figura 2: Resultados visuales experimento 3d-simple-media-alta, método Voxel

Estos resultados, en conjunto con la información de la Tabla 2, validan y expanden la teoría descrita anteriormente. En el experimento 3d-simple-media-alta, la figura h2_running_person presenta un área de 5.3% y una relación de 2.34, esta configuración geométrica afecta negativamente a aquellas siluetas cuyas proporciones difieren drásticamente. El caso más evidente es s2_rectangulo (área del 18.7% y aspecto de 1.55), el cual sufrió una caída considerable en su métrica de Recall (0.8758) a pesar de la simplicidad de su forma. En contraste, este efecto destructivo no se observa en m1_bird, que logra mantener un rendimiento estable y similar al baseline gracias a que su proporción de área (7.9%) es geoméricamente más compatible con las exigencias de vaciado de la silueta principal.

Por otro lado, al analizar los resultados de Mesh y comparar su baseline (Tabla 3) con los experimentos espejo (Tabla 4), se evidencia un comportamiento opuesto al de Voxel. En este método, los valores de Recall se mantienen estables (>0.94) y no sufren las caídas observadas en el IoU, es decir, el fenómeno descrito anteriormente no ocurre en este método. Esta retención de masa se explica por el funcionamiento descrito en el paper: a diferencia de Voxel, que opera tallando y eliminando bloques geométricos dentro de un volumen delimitado, Mesh funciona mediante la deformación continua de los vértices de una forma base. Por lo tanto, al enfrentarse a las siluetas de los experimentos espejo, la malla logra estirarse para alcanzar los contornos de la figura principal (la de menor área y mayor relación de aspecto), pero las funciones de regularización geométrica (Laplacian smoothing y edge length loss) actúan como fuerzas de cohesión que impiden el vaciado del centro tridimensional, como ocurre con Voxel.

Sin embargo, el forzar al mesh a cubrir completamente estas figuras de forma simultánea transforma el problema original de "omisión de masa" en un problema de "adición de masa extra". Geométricamente, la caída de rendimiento deja de estar ligada al Recall y pasa a reflejarse en

Precision. Esto queda evidenciado empíricamente al promediar las métricas de todos los experimentos espejo: mientras Voxel presenta una Precision del 97.9% y un Recall del 91.8%, Mesh invierte estos valores, registrando una Precision promedio del 90.4% con un Recall del 98.4%. Estos valores cruzados (diferencia de 7.5% a favor de Voxel en Precision, y de 6.6% a favor de Mesh en Recall) demuestra matemáticamente la transformación del fenómeno.

A partir de esto, se cree que este nuevo fenómeno, en vez de estar vinculado a áreas pequeñas o a la relación de aspecto extrema como ocurría en Voxel, está directamente determinado por la complejidad topológica de las siluetas evaluadas (presencia de agujeros, concavidades y extremidades separadas). Dado que la malla original posee una topología cerrada que no puede generar perforaciones o separar componentes físicos. Por lo tanto, para lograr alcanzar las extremidades dispersas de figuras altamente complejas (como el corredor o la mariposa), el algoritmo se ve obligado a generar "puentes" entre ellas, relleno con material sobrante espacios que deberían corresponder al fondo de la imagen, es decir, agregando masa extra al mismo tiempo que minimiza las omisiones de masa de las figuras. El mejor ejemplo de esta explicación es el experimento 3b-1media-2alta, en el que se usa a una figura topológicamente compleja como butterfly, tal como se muestra a continuación:

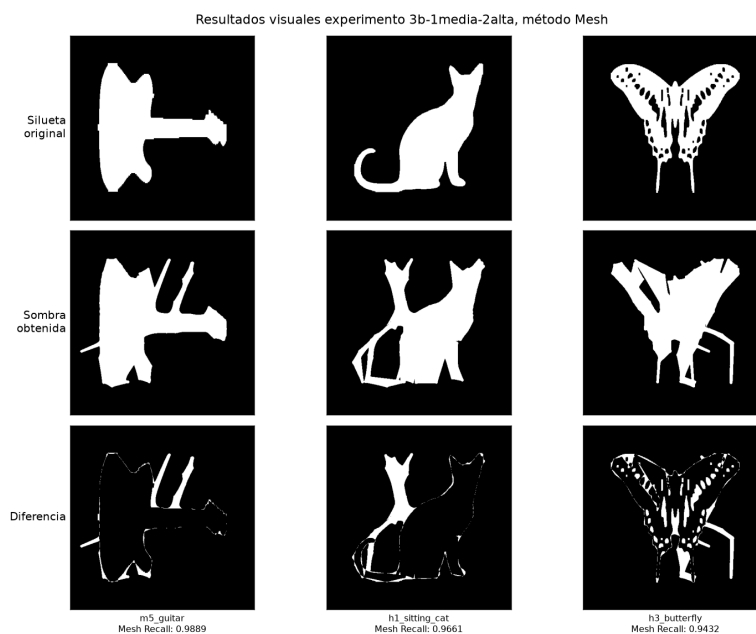


Figura 3: Resultados visuales experimento 3b-1media-2alta, método Mesh

Reconstrucción 3D

Como se explicó en la sección de datos, se ejecutó el pipeline para reconstruir el modelo 3D correspondiente a cada conjunto de siluetas.

Tanto para el modelo 'car' como 'rifle' se compararon sus resultados de acuerdo al método que se escogiese, ya sea una representación hecha de voxels o meshes, además se fue incrementando la cantidad de siluetas utilizadas para vislumbrar de qué manera cambia la calidad de la reconstrucción.

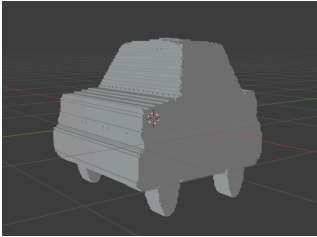
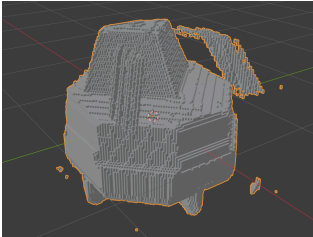
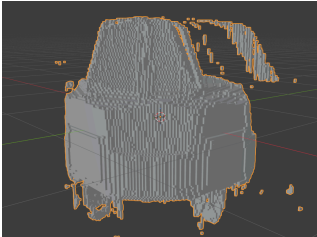
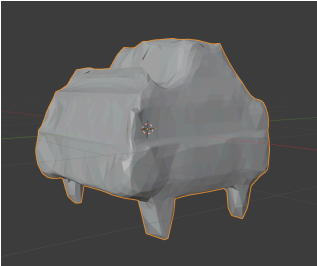
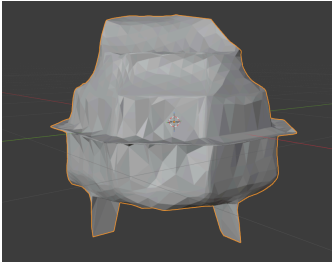
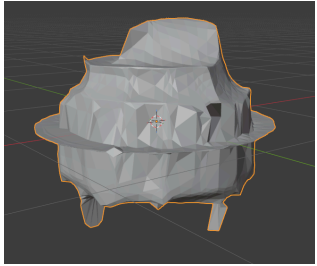
Voxel		
2 siluetas 	4 siluetas 	8 siluetas 
Mesh		
2 siluetas 	4 siluetas 	8 siluetas 

Tabla 6: Reconstrucción modelo Car, comparación de voxels y meshes.

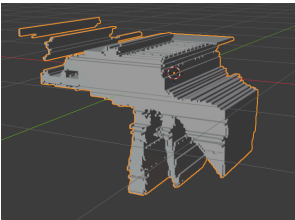
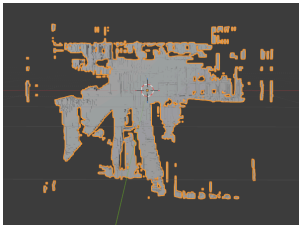
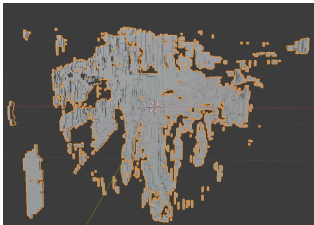
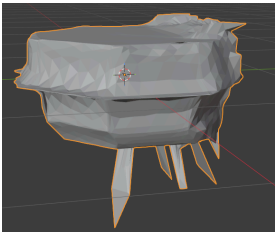
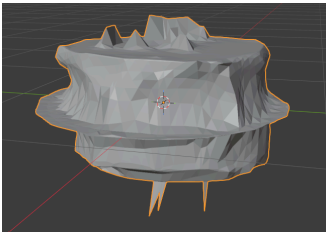
Voxel		
2 siluetas 	4 siluetas 	8 siluetas 
Mesh		
2 siluetas 	4 siluetas 	8 siluetas 

Tabla 7: Reconstrucción modelo Rifle, comparación de voxels y meshes.

En general se obtuvo que la calidad de las reconstrucciones es en sí baja, siendo la representación en voxel la que presentó los mejores resultados para los dos modelos evaluados. Al mirar las métricas de la tabla 5, una cosa que queda clara es que el pipeline se desempeña mejor para objetos más simétricos, concretamente favorece formas más cuadradas cuando se usa el método de voxels, lo cual tiene sentido, dado que en dicho flujo se parte de una grilla de voxels, donde cada uno

empieza con una densidad 1, lo cual resulta en un modelo inicial con una forma parecida a la de un cubo, debido a esto es que las reconstrucciones del auto presentan formas más parecidas a las del modelo original en cada una de sus iteraciones. Por otra parte el rifle es el que peor resultados tiene, lo cual es esperable pues tiene formas más irregulares, con muchos detalles a lo largo de su superficie que pueden ser complicados de detectar correctamente dadas las limitante que supone trabajar con sus siluetas, pues dada la naturaleza de las mismas algunos detalles se perderán definitivamente.

Todos los valores de IoU 3D en la tabla 5 no llegan ni a 0.1, y varios casos del rifle son 0, es decir, prácticamente no hay superposición de volumen entre lo reconstruido y el modelo real, incluso en los casos donde la reconstrucción se ve razonablemente parecida a simple vista, como el auto con 2 siluetas. Esto nos dice que aunque el modelo aprenda a proyectar sombras correctas no garantiza que haya aprendido a reconstruir la forma correcta del objeto en sí.

Sobre el IoU 3D del rifle, en el caso del voxel cae a exactamente 0 en los casos que usa 4 y 8 siluetas, mientras que el recall sube y la precisión se mantiene estable, lo que es un indicador de una reconstrucción dispersa y fragmentada, tal como se observa en la tabla 7, el voxel del rifle con más vistas se convierte en una nube de fragmentos desconectados en vez de una forma sólida. Muchos de esos fragmentos sueltos caen dentro de la tolerancia del 5% de algún punto de la superficie real, seguramente esa sea la razón del recall relativamente alto, pero al no formar una masa continua no llegan a ocupar el mismo volumen que el rifle real, lo que explica el IoU de 0.

En cuanto a las razones por las cuales las reconstrucciones fallan, se puede destacar el hecho que el pipeline espera que las siluetas proporcionadas respeten un patrón de orden y que todas sean capturadas a una misma distancia, porque durante el proceso de reconstrucción son tratadas como si entre ellas formaran un anillo que encapsula al objeto desde todos sus lados, entonces cuando se van agregando nuevas vistas, aumentan las probabilidades de que una imagen quede en la posición incorrecta, provocando que el optimizador intente satisfacer una restricción de silueta que es geoméricamente imposible de cumplir de forma consistente con el resto, luego todo lo descrito puede ser producto de un error durante el proceso de captura de los datos o un error al momento de dictar el orden de las siluetas para cada experimento, como se dijo con anterioridad, este fallo no se nota tanto si se usan pocas imágenes para un modelo que de base sea altamente simétrico como es el caso del auto, sin embargo frente a objetos más irregulares, las carencias se hacen cada vez más evidentes.

Conclusiones

Este proyecto evaluó la capacidad de generalización del método Shadow Art mediante una replicación sistemática del pipeline original, incorporando un nuevo dataset de 15 siluetas categorizadas por complejidad geométrica y un esquema de evaluación ampliado que incluyó métricas de ROI Pixel Accuracy, Precision y Recall.

Respecto a la generalización, la evaluación concluye que el desempeño depende intrínsecamente de la representación matemática utilizada. Ante el estrés geométrico, Voxel fracasa por una sustracción destructiva que mutila las formas al intentar reconciliar figuras incompatibles, mientras que Mesh fracasa por una adición no deseada que satura los espacios negativos de las formas complejas al no poder representar perforaciones.

Sobre la reconstrucción 3D, la evaluación directa contra los modelos originales de Blender mostró que un buen ajuste de sombras no implica una buena reconstrucción, todos los experimentos obtuvieron un IoU 3D bajo, llegando a 0 en varios casos del rifle. Esto evidencia la ambigüedad inherente del problema porque existen múltiples formas 3D capaces de proyectar las mismas sombras, y el método optimiza para que haya una consistencia 2D, sin ninguna garantía sobre la

geometría del objeto en sí. El método para las representaciones en voxel tiende a fragmentar el objeto en piezas dispersas, mientras que mesh acostumbra a generar "puentes" de material que alejan la forma resultante del original. A lo anterior se suma una limitación práctica, ya que el pipeline exige que las siluetas respeten un orden y una distancia de captura consistentes entre sí, y esta exigencia se vuelve más frágil a medida que se agregan vista, un problema que pasa desapercibido en objetos simples y simétricos como el auto, pero que se hace evidente en objetos con figuras más irregulares como la del rifle.

Las principales limitaciones del trabajo se centran en tres ejes críticos:

1. **Reproducibilidad:** La evolución de las librerías base (PyTorch3D, NumPy) y la ausencia de control determinista (seeds) en el código original impidieron una réplica exacta de los resultados del paper, obligando a establecer un baseline propio para asegurar la validez del análisis.
2. **Limitaciones Geométricas (Voxel):** Se demostró que siluetas con relaciones de aspecto extremas o áreas reducidas provocan una erosión de la masa central, la cual resulta indispensable para la conformación de otras proyecciones en el volumen.
3. **Limitaciones Topológicas (Mesh):** La estructura de malla cerrada actúa como un límite inmanente que impide generar perforaciones. Ante siluetas complejas, el modelo se ve forzado a crear "puentes" de material sobrante para alcanzar extremidades dispersas, lo cual degrada la precisión al añadir masa extra innecesaria.
4. **Limitaciones de los datos:** La reconstrucción 3D requiere de que los datos de un mismo objeto sigan un patrón de orden y distancia consistente, de modo que la captura de estas siluetas debe ser más extensa y exhaustiva, para considerar una mayor cantidad de perspectivas

Discusión transversal

El desarrollo de este proyecto permitió una comprensión profunda de las limitaciones intrínsecas de los métodos de renderizado diferenciable aplicados al shadow art. A continuación, se presenta una reflexión técnica sobre el proceso experimental:

1. **Aprendizajes:** La lección principal reside en la distinción fundamental entre representaciones discretas (Voxel) y continuas (Mesh). Aprendimos que el rendimiento no es una propiedad estática del modelo, sino que está condicionado por la geometría de la entrada, por ejemplo la representación Voxel es superior para capturar volúmenes densos, pero sufre ante "estrés geométrico", mientras que la topología cerrada de Mesh actúa como un regulador que prioriza la conectividad sobre la precisión de los bordes. Asimismo, reforzamos la importancia de métricas interpretables (como el ROI Pixel Accuracy) sobre indicadores genéricos para diagnosticar fallas específicas del optimizador.
2. **Dificultades técnicas:** La captura de los datos para la reconstrucción 3D fue complicada, dado que requirió en primera instancia familiarizarse con Render para poder procesar cada una de las imágenes extraídas de los modelos, también está el hecho de adaptar estas siluetas al formato esperado por el pipeline, si bien se escribió un script encargado de ajustar estos datos, el resultado no siempre fue el ideal por lo que fue necesario intervenir algunas imágenes de forma manual.
3. **Decisiones:** La propuesta original contemplaba utilizar la Figura 4 del paper como referencia directa. No obstante, al notar las discrepancias derivadas del cambio de entorno, decidimos descartar esa comparación directa y establecer nuestro propio baseline. Esta decisión fue crucial, ya que nos permitió comparar los resultados bajo las mismas condiciones de hardware y software, otorgando validez estadística a nuestras conclusiones.

4. Funcionamiento:

- a. **Mejor de lo esperado:** Superó nuestras expectativas la capacidad de la métrica ROI Pixel Accuracy para aislar errores de fondo de errores de silueta, facilitando la identificación del problema de "masa extra" en Mesh.
- b. **Peor de lo esperado:** Por el contrario, fue peor de lo esperado la carga computacional y la sensibilidad de Voxel, debido a que los tiempos de ejecución excedieron las 4 horas por experimento, lo que limitó nuestra capacidad de iterar sobre más configuraciones. Además, subestimamos el impacto destructivo de la erosión de la masa central en Voxel cuando se procesan figuras de baja compacidad.

5. Continuación del proyecto:

De continuar el proyecto, implementarían tres cambios fundamentales:

- a. **Control de aleatoriedad:** Desarrollar un wrapper para forzar el determinismo en las operaciones de GPU, buscando replicar con mayor fidelidad los estados del optimizador.
- b. **Regularización dinámica:** Para el método Mesh, explorar funciones de pérdida adaptativas que penalicen la creación de "puentes" topológicos, permitiendo al modelo realizar incisiones o perforaciones.
- c. **Preprocesamiento normalizado:** Implementar una normalización de siluetas basada en centro de masa y relación de aspecto para mitigar el sesgo inducido por figuras con aspect ratios extremos.

Distribución de tareas

- Erick Lara:
 - Informe: Datos, hasta limitación del conjunto de siluetas generadas. Evaluación, Hasta la figura 3.
 - Código Jupyter del método de Voxel, realización de los experimentos con Voxel.
- Nicolás Medel Correa:
 - Informe: Evaluación, conclusiones, discusión
 - Código: Obtención de siluetas propias, segmentación de siluetas propias, código Jupyter del método de Mesh, realización de los experimentos con Mesh.
- Vicente Retamal:
 - Informe: Datos, evaluación, conclusiones relacionadas a la reconstrucción 3D
 - Código: Obtención de siluetas propias a partir de modelos 3D procesados en Blender, código Jupyter para la reconstrucción de modelos tanto para mesh como para voxel.

Uso de IA

- Erick Lara:
 - Modelo(s): Gemini 3.5 Flash
 - Preguntas: Presencia de bugs en el repositorio de los autores, funcionamiento del método Voxel, definición de las métricas IoU y Dice, fórmulas de IoU, Dice, Precision, Recall y ROI PA.
 - Partes generadas: Código de Voxel_method.ipynb (No se modificó el pipeline de los autores, solo se agregaron los experimentos, 3 métricas nuevas, una función para evaluar las métricas y una sección para mostrar y guardar los resultados). Se reescribieron algunas secciones del informe para mayor claridad, manteniendo la idea original sin agregar nada nuevo.

- Estrategia de validación: Se verificó que los experimentos generados fueran los indicados, y luego se confirmó su correctitud a través del modelo 3D generado para 2 siluetas simples. También se verificó que la métrica indicada (ROI Pixel Accuracy) funcionara como se indicó a través de experimentos con figuras simples.
- Modificaciones al output: Se simplificó aún más la reescritura generada por la IA, se encontró un bug en la automatización de los experimentos en el notebook y se corrigió (sobrescritura de los resultados).
- Nicolás Medel Correa:
 - Modelo(s): Claude Sonnet 5 y Gemini 3.5 Flash
 - Preguntas: Funcionamiento del método Mesh, definición de las métricas IoU y Dice, fórmulas de IoU, Dice, Precision, Recall y ROI PA.
 - Partes generadas: Segmentación de nuevas imágenes, código de Mesh_method.ipynb, para replicar, visualizar y cachear los resultados del pipeline con los datos base y antiguos. Se reescribieron algunas secciones del informe para mayor claridad, manteniendo la idea original sin agregar nada nuevo.
 - Estrategia de validación: Se verificó que las imágenes segmentadas estuvieran binarizadas de la misma forma, y que el pipeline tuviera los mismos pasos que el paper original, respecto a mesh, y se comparó los resultados base con los obtenidos en el paper.
 - Modificaciones al output: Se iteró sobre lo escrito por IA sobre todo en la visualización de resultados, para poder tener métricas representativas.
- Vicente Retamal:
 - Se utilizó Claude Sonnet 5 para el desarrollo del notebook 3D_recons_metrics.ipynb encargado de registrar el IoU 3D, la precisión y recall de cada una de las reconstrucciones generadas. Se evaluó el funcionamiento de este script al comparar cada modelo original consigo mismo y con versiones ligeramente alteradas de estos.

Anexo

Modelo	Método	N° de siluetas	IoU 3D	Precision@0.05	Recall@0.05
Car	Voxel	2	0.0328	0.1088	0.1692
		4	0.0268	0.1942	0.1961
		8	0.0385	0.2311	0.2273
	Mesh	2	0.0287	0.0883	0.1363
		4	0.0172	0.1218	0.1719
		8	0.0164	0.1096	0.1682
Rifle	Voxel	2	0.0061	0.0791	0.5259
		4	0	0.1099	0.6112
		8	0	0.1017	0.6283
	Mesh	2	0.0069	0.0176	0.0973
		4	0.0005	0.0156	0.0620

		8	0	0.0219	0.1102
--	--	---	---	--------	--------

Tabla 5: Métricas de desempeño de la reconstrucción 3D.

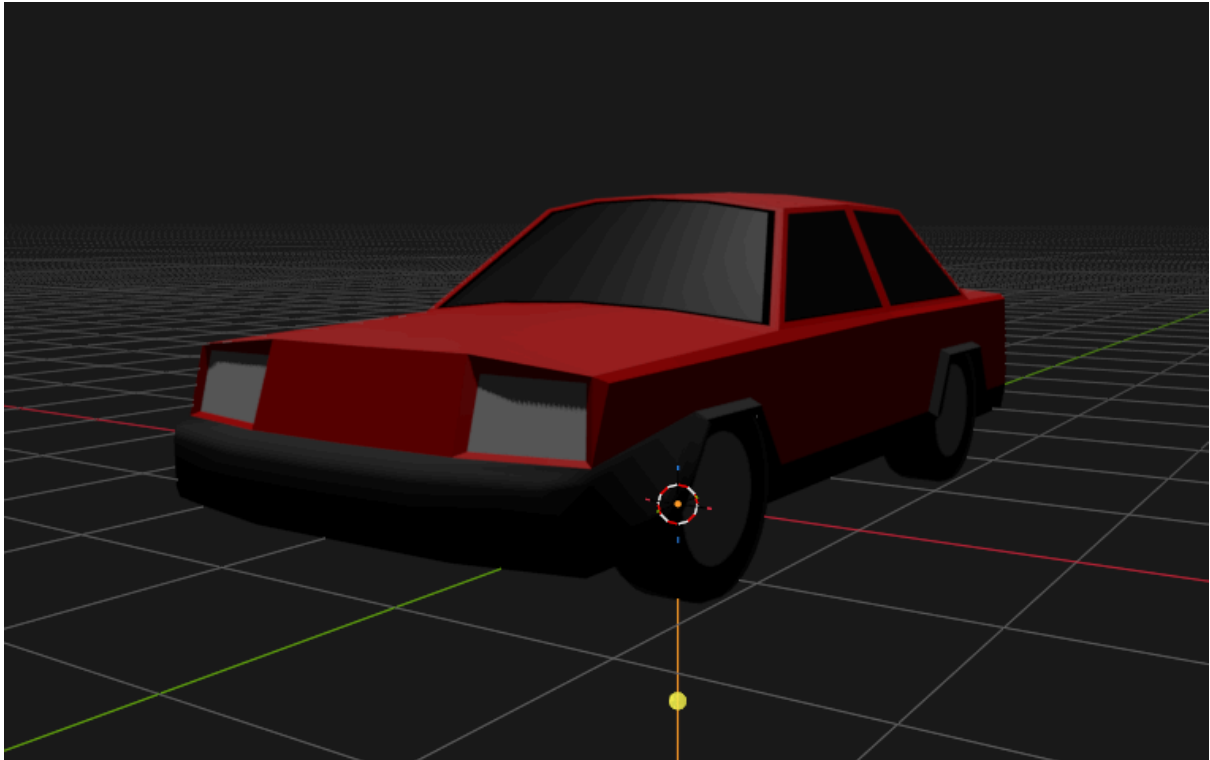


Figura 4: Modelo Car original.



Figura 5: Modelo rifle original